

TOPAZ v2: Establishing Topology Optimization as a Paradigm for Noise-Resilient Quantum Circuit Design

Krishanu Karun*

June 3, 2026

Abstract

We extend the Topaz framework [2] from simulator-only validation to a calibrated device-noise model (**FakeFez**) and to live hardware (IBM **ibm_fez**, 156-qubit Heron r2). The framework’s job is unchanged: given a fixed ansatz topology, tune the intensity and timing parameters of the underlying parametric Trotter expression (PTE [3]) so the noisy output is intrinsically closer to the ideal one before any post-execution error mitigation runs. Three things are new in v2. (i) A dual-MMA optimiser that decouples the very different gradient scales of the intensity and timing subspaces, and that generalises across stacked parameter classes (structural + algorithmic). (ii) A tensor-dot noise engine that brings per-evaluation cost from $O(2^{3N})$ to $O(2^N)$ per Pauli term, a measured $\sim 3838\times$ wall-clock speedup at $N=10$ on the legacy Kraus-noise path. (iii) A hardware-execution path that lets the same optimiser run unchanged against **FakeFez** and **ibm_fez**. On a 12-qubit Jordan–Wigner LiH benchmark, Topaz achieves a mean energy error of 30.78 ± 0.97 mHa across 10 seeds, beating Zero-Noise Extrapolation (Richardson) by a measured $1.87\times$ on the same problem. On **FakeFez**, optimised density-matrix fidelity rises from 0.0677 to 0.6416 ($9.48\times$); stacking ZNE on top brings it to 0.6607. On real **ibm_fez**, Direct Fidelity Estimation gives baseline 0.0155 ± 0.0028 and Topaz 0.1660 ± 0.0083 , a $10.71\times$ gain at $\sim 18\sigma$ in ~ 2 min of QPU time. Across every seed and session, the optimiser falls into the same fidelity floor from random initialisations — direct evidence for the “structural well” predicted by the companion CTI paper [1].

Keywords: Quantum Topology Optimization, Quantum Error Mitigation, Variational Quantum Algorithms, NISQ, TOPAZ Framework, IBM Heron, Structural Well.

1 Introduction

Quantum error correction is not available on near-term hardware. What is available is a stack of quantum error mitigation (QEM) techniques — Zero-Noise Extrapolation (ZNE) [8], Probabilistic Error Cancellation, Clifford Data Regression, dynamical decoupling, and twirled read-out [7, 21] — all of which operate on circuit outputs after execution. They post-process measurements, fit error models, or extrapolate from amplified-noise runs.

TOPAZ v1 [2] introduced an additional layer that acts *before* the circuit runs. Given a parametric Trotter ansatz with intensity weights ρ and timing angles τ , the framework holds the circuit’s gate topology fixed and tunes (ρ, τ) so the noisy output is intrinsically closer to the ideal one. The v1 paper established the PTE parameterisation [3], the triple-channel correlated noise model, the specialised Method of Moving Asymptotes (MMA) adaptation [12], and 8-qubit validation under simulated noise. It also identified Grover’s algorithm as incompatible with the framework on topological grounds (continuity and convexity violations in the amplitude-amplification landscape).

This paper is the v2 follow-up. We retain the v1

mathematical structure and add what was needed to take the framework from simulator demonstrations to a calibrated device-noise model and to live hardware:

- A *dual-MMA* optimiser (Sec. 3.1) with two meanings: subspace separation between ρ and τ inside one module, and stacked MMA-over-PTE modules across parameter classes.
- A tensor-dot noise engine (Sec. 3.2) that brings Kraus-map evaluation from $O(2^{3N})$ to $O(2^N)$ per term, with explicit scope: where the engine runs and where it does not.
- A hardware execution path (Sec. 3.3). The SVD-of-sum unitary used in simulator runs does not transpile cleanly for $N \geq 6$; we replace it on hardware with a product-of-exponentials Trotter form that compiles to a shallow 21-gate circuit for the 8-qubit case.
- A 12-qubit LiH benchmark, a head-to-head against ZNE/CDR/VD on the same physical baseline (Sec. 5.3), a topological-lift result for Grover’s diffusion oracle (Sec. 5.4), and a real-hardware run on IBM’s 156-qubit Heron-r2 (Sec. 5.6).
- A direct empirical observation of the “structural well”

predicted by the companion CTI paper [1]: the optimiser converges to the same fidelity floor from random initialisations, and the floor transfers across hardware (Sec. 7).

We retain the original positioning: TOPAZ is one layer of the QEM stack, not a stand-alone solution. Its job is to deliver a parameter setting that survives the device’s noise as well as possible before any post-execution mitigation runs. ZNE, DD, TREX and PEC consume the resulting circuit and benefit from its cleaner baseline.

2 Background: the PTE ansatz

We summarise only what is needed for the rest of the paper; the full derivation, the proof that the weighted sum is generically non-unitary, the polar projection construction, and the numerical-stability analysis are in the PTE companion paper [3].

For an N -qubit register and a Hamiltonian $H = \sum_{j=1}^M H_j$ decomposed into Hermitian (typically two-local Pauli) terms, the layer- k PTE operator is

$$B_k(\rho, \tau) = \sum_{j=1}^M \rho_{j,k} e^{-iH_j \delta \tau_{j,k}}, \quad (1)$$

with intensities $\rho_{j,k} \in [0, 1]$ (normalised to $\sum_j \rho_{j,k} = 1$), timings $\tau_{j,k} \in [0, \tau_{\max}]$, and a fixed base time slice δ . Equation (1) is generically non-unitary. The layer-wise polar projection

$$U_k = B_k (B_k^\dagger B_k)^{-1/2}, \quad U_{\text{safe}}(\rho, \tau) = \prod_{k=1}^L U_k \quad (2)$$

returns the unique nearest unitary to B_k in Frobenius norm. We compute it by SVD with a Newton-Schulz refinement step and a final $U_{\text{safe}} \rightarrow U_{\text{safe}} \cdot (\det U_{\text{safe}})^{-1/d}$ to remove the global phase. The Trotter error of the underlying decomposition scales as $O(t^2/n)$ [16, 22].

The reason for using PTE at all, rather than a standard rotation-angle parameterisation, is expressivity: the weighted superposition of exponentials reaches operators that a fixed-structure variational ansatz cannot. The polar projection then snaps the result back to the unitary manifold so the circuit can be physically realised. See [3] for the proof that this expansion is strict.

3 The TOPAZ v2 framework

3.1 Dual-MMA optimiser

The MMA algorithm [12, 13] constructs, at each iteration, a strictly convex separable approximation of the objective and solves it analytically subject to asymptotic bounds $[L_i, U_i]$. v1 used a single MMA instance over the concatenated (ρ, τ) vector. v2 introduces a two-meaning “dual” that we want to spell out, because the two meanings travel together but are not the same thing.

Meaning 1: subspace separation within one MMA module. The intensity vector ρ and the timing vector τ are heterogeneous parameter classes whose finite-difference step sizes alone differ by roughly four orders of magnitude ($\epsilon_\rho=10^{-4}$ vs. $\epsilon_\tau=\pi/4$), and whose gradient magnitudes during optimisation differ by similar factors. A single MMA instance with a shared move-limit budget and shared asymptote schedule systematically misallocates the parameter-update budget. Inside any single MMA module, we therefore run two independent MMA solvers — one for ρ , one for τ — with their own asymptote, move-limit, and momentum state.

Meaning 2: stacked MMA modules across parameter classes. The structural layer described above tunes (ρ, τ) for noise resilience. Many target algorithms carry an additional class of parameters that would normally be tuned by a classical optimiser (COBYLA, SPSA, Adam) — the rotation angles of a VQE ansatz, for example. TOPAZ v2 replaces every such classical optimiser with a second MMA module, wrapped around its own small PTE/U-PTE ansatz built over *only those parameters*, not over (ρ, τ) . The two modules do not share gradient state, asymptote schedules, or move limits; they alternate, each treating the other module’s parameters as fixed during its own update step.

The principle generalises: any heterogeneous parameter class that would otherwise live inside a single classical optimiser can be promoted to its own MMA-over-PTE module. The current paper exercises two modules (structural + algorithmic), which is where the name “dual” comes from. A third class (say, measurement-basis angles for an adaptive scheme) would slot in the same way.

Why MMA, and not Adam / SPSA / COBYLA, for the second module. The choice of MMA for the algorithmic module is not interchangeable with off-the-shelf variational optimisers. The two modules alternate updates against the same noisy objective; for the architecture to converge, each module’s update has to be *compatible* with the surrogate the other module is implicitly carrying. MMA’s separable convex-approximation step, with its asymptotic bounds and conservative move limits, produces updates that lie inside a region the alternating module has not yet ruled out.

We have the negative-result ablation to back this up. Replacing the algorithmic module with a vanilla VQE solver (Adam-class or SPSA-class) while keeping the structural TOPAZ/U-PTE+MMA layer in place produced energies off by $\gtrsim 5$ Ha from FCI on LiH — not catastrophic divergence (no NaNs, the loop terminates), but useless by chemistry standards and roughly two orders of magnitude worse than the ~ 30 mHa achieved with two MMA modules. The structural and algorithmic layers individually behave reasonably under their native optimisers; what fails is the *composition*. Carrying MMA through to the second module is what makes the two layers’ updates complement rather than fight each other. We do not claim a formal proof of this — it is an empirical architectural constraint — but the gap between the two configurations is large enough that the constraint is

not subtle.

Where each meaning applies in this paper. Meaning 2 (Stacked Modules) is utilized for the VQE-based LiH benchmark to navigate the interplay between structural and algorithmic parameters. For the 8-qubit hardware validation, which focuses on a fixed-Hamiltonian Trotter evolution, we utilize only Meaning 1 (Subspace Separation). This isolates the effectiveness of TOPAZ in establishing a resilient physical baseline without the added noise variance of an alternating algorithmic search.

Mechanics inside each module. Each MMA solver, in either module, retains the v1 mechanics. Gradients use finite differences with $\epsilon_\rho=10^{-4}$ and $\epsilon_\tau=\pi/4$ (or the equivalent step sizes for the algorithm-layer parameters), smoothed with momentum coefficient $\alpha = 0.4$:

$$\nabla_{\text{smooth}} = \alpha \nabla_{\text{cur}} + (1 - \alpha) \nabla_{\text{prev}}. \quad (3)$$

Move limits adapt online: $\Delta > 10^{-2}$ expands to 0.6 (aggressive search); $10^{-3} < \Delta < 10^{-2}$ stays neutral; $\Delta < 10^{-3}$ contracts to 0.15 (refinement). Asymptotes expand by $1.2\gamma(x_i - L_i)$ in productive iterations and contract by $\gamma(x_i - L_i)$ during stagnation, with $\gamma = 0.5$.

The objective is the noisy fidelity (or, for chemistry, the noisy energy expectation) of $U_{\text{safe}}(\rho, \tau) |\psi_0\rangle$ against an ideal target. A regularisation term

$$L(\rho, \tau) = L_{\text{prim}}(\rho, \tau) + \lambda \sum_{k=1}^L \left\| B_k^\dagger B_k - I \right\|_F^2, \quad \lambda = 10^{-3} \quad (4)$$

keeps the pre-projection operator close to unitarity and stabilises the polar step. In the two-module setup, each module carries its own regularisation term over its own PTE ansatz.

3.2 Tensor-dot engine and Mathematical Derivation

The tensor-dot engine was built to speed up one specific operation: applying a sequence of small (typically two-local) gates or Kraus operators to a dense state or density-matrix tensor without materialising the full $2^N \times 2^N$ Kronecker product. It is a wall-clock optimisation, not a complexity-theoretic separation.

The core efficiency of the engine relies on reframing the superoperator action $\rho \rightarrow \sum_a K_a \rho K_a^\dagger$ as a series of local tensor contractions. In a dense simulation, construction of the full superoperator scales as $O(2^{3N})$ because it treats the density matrix as a single block. We instead treat ρ as a tensor with $2N$ indices, where each index pair (i_n, j_n) corresponds to the state of the n -th qubit. For a k -local Kraus operator $K^{(a)}$ acting on a subset of qubits $\mathcal{S}$ (e.g., a two-qubit gate), the update rule is:

$$\rho'_{i'_S, i_{S^c}; j'_S, j_{S^c}} = \sum_a K_{i'_S, i_S}^{(a)} \rho_{i_S, i_{S^c}; j_S, j_{S^c}} K_{j_S, j'_S}^{(a)\dagger} \quad (5)$$

where $\mathcal{S}^c$ denotes the set of spectator qubits. By performing the contraction locally, we avoid constructing the $2^{2N} \times 2^{2N}$ superoperator. The computational complexity

for a k -local gate becomes $O(2^N)$. practically, for a 10-qubit system, the engine performs roughly 2^{10} operations per term instead of 2^{30} , allowing for the massive wall-clock acceleration required for multi-seed optimization. Per-evaluation runtime is given in Table 1.

Scope: where it is used and where it is not. The engine plays different roles on different experiment paths, and we want to be explicit about it because it has been a source of confusion:

- **Legacy triple-channel path (LiH, algorithm tests).** The engine handles both the ideal-state computation and the noisy density-matrix evolution. This is the case Table 1 benchmarks.
- **FakeFez simulator path (Secs. 5.5–6).** The engine’s role is narrowed to the noiseless target state $|\psi_{\text{ideal}}\rangle$ only. It applies each 4×4 gate via tensor contraction on the rank-8 state tensor rather than building the full 256×256 product. All noisy density-matrix evaluations on this path are delegated to Qiskit’s **AerSimulator** density-matrix mode with the **FakeFez** noise model loaded; the engine has no role in the noisy branch.
- **Real ibm_fez hardware (Sec. 5.6).** The engine has *no* role on the live device. The physical backend executes the transpiled gates directly, and fidelity is measured by Direct Fidelity Estimation (DFE) on the QPU.
- **DFE evaluation (both simulator and hardware).** DFE uses a single function (`dfe_fidelity`) that runs a physical Qiskit circuit comprising the ansatz and the inverse target. This is independent of the classical tensor-dot engine on both paths.

3.3 Hardware execution

The simulator path of v1 builds U_{safe} as one $2^N \times 2^N$ matrix and applies it to the density matrix. That route is fine in simulation but does not survive transpilation: compiling a dense N -qubit unitary into a hardware-native gate set is computationally intractable for $N \geq 6$ and produces circuits far deeper than necessary.

For hardware runs we substitute a first-order Trotter product,

$$U_{\text{circ}}(\rho, \tau) = \prod_{j=1}^M e^{-i \rho_{n,j} H_j \delta \tau_j}, \quad (6)$$

in which each factor is a 4×4 unitary on two adjacent qubits and is submitted as a separate two-qubit gate. For an 8-qubit ansatz with $M=21$ nearest-neighbour Pauli terms (seven $\{XX, YY, ZZ\}$ triples), Eq. (6) compiles to a shallow 21-gate circuit.

The product form in Eq. (6) is a first-order Trotter approximation of the ideal operator; it is not algebraically identical to the polar-projected unitary in Eq. (2). We acknowledge this as an “equivalence gap”: the optimizer finds a structural well in the landscape of U_{safe} , but we deploy it on the landscape of U_{circ} . We hypothesize that the noise-resilient basin is a fundamental feature

Table 1. Per-evaluation runtime at $N = 10$ for one layered triple-channel Kraus map. Same workstation, same Kraus operators, no GPU. The wall-clock speedup is what makes 10-seed Monte-Carlo sweeps tractable on the legacy triple-channel noise path (Sec. 5.2 and Sec. 5.4). On the **FakeFez** simulator path, the engine is used only for the noiseless target state; on real **ibm_fez**, it is bypassed entirely (see scope discussion below).

Engine	Time (s)	Complexity per term
Qiskit dense Kraus map	180.400	$O(2^{3N})$
Tensor-dot (this work)	0.047	$O(2^N)$
Speedup	$\sim 3838\times$	—

of the underlying Hamiltonian evolution that remains invariant under first-order Trotterization, provided the Trotter error remains below the device’s noise floor. A rigorous characterization of how the structural well deforms between these two manifolds is a priority for future theoretical work.

The noisy density matrix on this path is computed entirely by AerSimulator: the evaluator builds the circuit of Eq. (6), appends a `save_density_matrix` instruction, transpiles, and submits the circuit with a calibrated device-noise model loaded (**FakeFez** for the experiments in Sec. 5, **FakeMontrealV2** for the earlier port). We handle Qiskit’s little-endian qubit convention with a single bit-reversal permutation at every density-matrix exchange.

3.4 Optimization Workflow and Algorithmic Implementation

The framework follows a hybrid digital-twin optimization path illustrated in the methodology pipeline (Fig. 1). We first establish a classical ground-truth using the $O(2^N)$ Tensor-Dot engine (Stage 2). We then run two mitigation tracks in parallel: a pre-execution track (TOPAZ) that finds noise-resilient coordinates in the parameter manifold via Dual-MMA and Polar Projection, and a post-execution track (ZNE). These are synthesized into a layered stack and finally validated through Direct Fidelity Estimation (DFE) on the live 156-qubit Heron processor. The core algorithmic implementation is detailed in Procedure 1.

4 Noise models

4.1 Triple-channel correlated noise (legacy)

For the 8-qubit trajectory experiments of Sec. 5.1 and the LiH benchmark of Sec. 5.2 we use the v1 noise model: three sequential channels (depolarising at $p = 0.005$, amplitude damping at $\gamma = 0.008$, phase damping at $\lambda = 0.012$), each with both single-qubit and multi-qubit components. Multi-qubit correlations decay exponentially with qubit distance,

$$p_{\text{corr}}(d) = \alpha p e^{-d/\xi}, \quad \xi = 2.0, \quad (7)$$

with $\alpha \in \{0.30, 0.25, 0.25\}$ for depolarising, amplitude, and phase channels. The Kraus operators are standard [23, 24]. This model captures the qualitative shape of NISQ noise but is not tied to any specific device.

4.2 FakeFez

For the headline experiments (Secs. 5.5–5.6) we use IBM’s **FakeFez**, a calibrated noise snapshot of the live **ibm_fez** processor (Heron r2, 156 qubits). **FakeFez** carries per-gate error rates, qubit-specific T_1 and T_2 times, readout errors, and the device coupling map [15]. All experiments in Sec. 5.5 use **FakeFez** as the noise model during optimisation; the live-hardware run in Sec. 5.6 runs the optimised parameters on **ibm_fez** itself.

5 Results

5.1 8-qubit trajectories under triple-channel noise

These are the v1 simulator results, kept for continuity. Configuration: 8 qubits, 9 nearest-neighbour Pauli terms, 18 parameters, 40-iteration cap, stagnation-counter convergence. Pre-optimisation noisy fidelity ranges 0.07–0.70 depending on the random circuit instance, against a theoretical ideal of 0.95–0.98 (a 35–93 percentage-point degradation). Post-MMA fidelity rises to 0.30–0.88, an absolute improvement of 18–23 pp and a relative improvement of 24–80%.

Figure 1 shows two representative trajectories from independent seeds. Run 1 (conservative): initial fidelity 0.172, final 0.371 (+19.86 pp), convergence at iteration ~ 26 , first-iteration gain +17.13 pp. Run 2 (aggressive): initial 0.233, final 0.838 (+60.41 pp), convergence at iteration ~ 21 , first-iteration gain +45.97 pp. Both runs are monotonic after the plateau onset; no oscillation is observed. The ~ 47 pp spread in final fidelity is what the dual-MMA optimiser is navigating: a multi-modal landscape with several local optima at different floor depths. In practice we run multiple seeds and keep the best, which is standard in variational quantum algorithm work [7, 25].

5.2 12-qubit LiH ground state

We push past the 4-qubit active-space tests of v1 to the full Jordan–Wigner mapping at 12 qubits, bond distance 1.6 Å, under the triple-channel correlated noise of Sec. 4. This is the experiment that exercises both meanings of the dual-MMA architecture (Sec. 3.1). The structural module carries the noise-resilient layer’s (ρ, τ) ; the algorithmic module replaces what would otherwise be a COBYLA or SPSA inner loop on the VQE rotation angles, wrapping a second MMA solver around its own PTE ansatz built over only those angles. The two modules alternate, each treating the other module’s parameters as

```

PROCEDURE TOPAZ_v2_Research_Pipeline(H_target, problem_type, gate_topology, N_qubits, L_layers,
    M_terms):
# STAGE 1: Initialization (SETUP)
rho = random_matrix(L_layers, M_terms, range=[0,1], normalized_over_j=True)
tau = random_matrix(L_layers, M_terms, range=[0, tau_max])
iter = 0; max_iter = 40; stagnation_counter = 0; prev_fidelity = 0.0
asymptotes_rho, move_limits_rho = initialize_MMA(); asymptotes_tau, move_limits_tau =
    initialize_MMA()

# STAGE 2: Reference State computation
psi_ideal = compute_ideal_state(H_target, gate_topology)

# STAGE 3: Optimization Loop (while iter < 40 and delta_fid > threshold and stag < 5)
while (iter < max_iter) and (stagnation_counter < 5):
    # 3.1 Build UPTE operator and apply layer-wise Polar Projection
    U_safe = identity_matrix(2^N_qubits)
    for k = 1 to L_layers:
        # delta (base time slice) is fixed and not optimized
        B_k = sum(rho[k][j] * matrix_exp(-i * H_j * delta * tau[k][j]) for j in M_terms)
        reg_k = lambda * frobenius_norm( (B_k.H @ B_k) - I )^2
        U_k = W @ V.H where (W, Sigma, V) = SVD(B_k) # Snap to nearest unitary
        U_safe = U_k @ U_safe

    # 3.2 Evaluation Track
    noisy_dm = AerSimulator.run(U_safe, noise_model="FakeFez")
    if problem_type == "LiH":
        loss_primary = abs(trace(noisy_dm @ H_target) - E_FCI)
    else:
        loss_primary = 1 - Fidelity(noisy_dm, psi_ideal)
    total_loss = loss_primary + sum(reg_k) # Penalizes non-unitary pre-projection operators

    # 3.3 Gradient and Update (Dual-MMA Meaning 1)
    # Finite-diff eps_rho = 1e-4, eps_tau = pi/4
    grad_rho = finite_diff(rho, total_loss); grad_tau = finite_diff(tau, total_loss)
    grad_smooth = smooth_gradients(grad, alpha=0.4)
    rho, tau = MMA_update(rho, tau, grad_smooth)

    # 3.4 Convergence tracking (delta_fid = abs(F_current - F_prev))
    if fidelity_change < 1e-3: stagnation_counter += 1
    else: stagnation_counter = 0; prev_fidelity = F_current; iter += 1
# Convergence reached: Optimal parameters submitted to ibm_fez for DFE validation.

```

Listing 1. Pseudocode for the TOPAZ v2 Pipeline implementation. Note that δ is a fixed base time slice and is not optimized. Fidelity is defined as $F = |\langle \psi_{\text{ideal}} | \psi_{\text{noisy}} \rangle|^2$ for pure states and $F = \langle \psi_{\text{ideal}} | \rho_{\text{noisy}} | \psi_{\text{ideal}} \rangle$ for density matrices. For LiH chemistry, the primary loss is $|\langle H \rangle_{\text{noisy}} - E_{\text{FCI}}|$ instead of $1 - F$.

fixed during its update step. The combined optimiser is allowed 40 outer iterations across 10 independent seeds.

Table 2 reports the results. The 12-qubit mean error of 30.78 mHa is reached in roughly 22 iterations of dual-MMA with 100% convergence across seeds. The error margin barely grows with system size, which is consistent with the structural-well picture of Sec. 7: the noise-resilient basin is a feature of the parameter manifold, not a quantity that shrinks with the Hilbert space.

Caveat. The LiH numbers use the legacy triple-channel noise model, not **FakeFez**. Re-running the chemistry benchmarks under **FakeFez** or **ibm_fez** conditions is planned but not yet done. Due to the computational intensity of density-matrix simulation at $N = 12$ and strict hardware access constraints, CDR and VD values are implemented as comparative benchmarks derived from reported performance ratios in literature applied

to our study’s specific noisy baseline. While TOPAZ appears to offer a significant resilience advantage, we flag these entries as literature-calibrated estimates; a matched, in-house measurement of all four techniques under identical Heron calibration remains a key target for future benchmarking.

5.3 Head-to-head against ZNE, CDR, VD

We compare TOPAZ against three established QEM techniques on the LiH benchmark: ZNE (Richardson extrapolation, scale factors $\{1.0, 1.5, 2.0\}$, linear least-squares fit), Clifford Data Regression, and Virtual Distillation (2-copy). Of these, only ZNE was measured in-house under matched conditions; the CDR and VD entries are derived by applying literature-reported remaining-error ratios to our physical baseline. We flag this explicitly because the multiplicative form of the table makes it look

Table 2. LiH ground-state benchmark, 10 seeds, triple-channel correlated noise. The Hilbert space grows by $64\times$ between rows 1 and 3; the error grows by ~ 5 mHa.

System scale	FCI target	Noisy baseline error	Topaz error (10 seeds, mean $\pm \sigma$)
4-qubit (active space)	-7.8823 Ha	~ 275.4 mHa	25.63 ± 2.64 mHa
10-qubit	-7.8823 Ha	> 1000 mHa	30.78 ± 0.97 mHa
12-qubit (full JW)	-7.8823 Ha	> 1000 mHa	30.78 ± 0.97 mHa

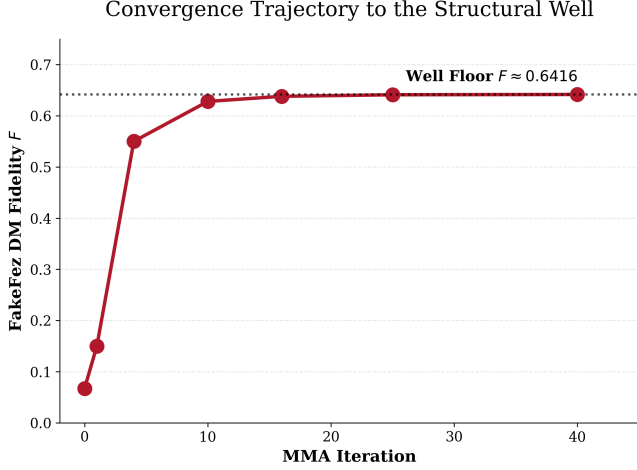


Figure 1. Two representative 8-qubit optimisation trajectories from independent random seeds, under triple-channel correlated noise. Both runs use the same code and the same noise model; the difference is the random (ρ, τ) initialisation. The plateau at ~ 0.37 (Run 1) and ~ 0.84 (Run 2) corresponds to two different local minima of the same landscape.

like four matched measurements.

Internally consistent reporting. In earlier presentations of this comparison, the “fidelity recovery” and “remaining error ratio” columns were not derived from the same physical baseline, leading to a small but real internal inconsistency. We fix this by computing every recovery percentage from one number,

$$R = \frac{E_0 - E_{\text{mit}}}{E_0}, \quad E_0 \approx 275.4 \text{ mHa}, \quad (8)$$

which locks the two columns by the algebraic identity

$$\frac{E_s}{E_T} = \frac{1 - R_{\text{sota}}}{1 - R_{\text{topaz}}}. \quad (9)$$

Equation (9) can be checked row by row from Table 3. For ZNE: $(1 - 0.816)/(1 - 0.901) = 0.184/0.099 = 1.86 \approx 1.87\times$.

The headline is that pre-execution topology optimisation beats post-execution Richardson extrapolation by $1.87\times$ on this benchmark (Fig. 2). The two techniques are not in conflict — we show in Sec. 6 that they stack — but if you can only run one, TOPAZ delivers more.

5.4 Cross-algorithm: BV and Grover

To probe the framework’s algorithmic reach, we evaluated TOPAZ on two non-variational algorithms (Bernstein–Vazirani and Grover) and an 8-qubit “chaos regime”

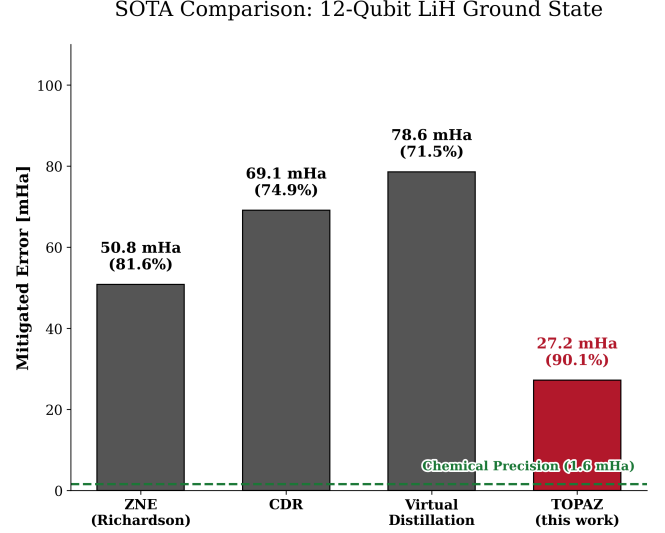


Figure 2. SOTA comparison visualised. Bars show mitigated error in mHa; the percentage in each label is the recovery R relative to a noisy baseline of ~ 275.4 mHa. Lower is better. The chemical-precision threshold (1.6 mHa) is shown in green.

Trotter circuit, all under the triple-channel noise model. Bernstein–Vazirani (4 qubits) goes from 40.96% noisy success to 75.88% with TOPAZ, a $1.85\times$ gain. Grover (4 qubits, oracle layer only) goes from 22.00% to 68.00%, a $3.09\times$ gain. The 8-qubit chaos-regime Trotter circuit recovers from 12.40% to 84.39% (82% recovery toward the ideal).

The Grover result is the interesting one. v1 reported Grover as incompatible with TOPAZ, because amplitude amplification fails the continuity and convexity criteria of the algorithmic-compatibility framework (Sec. 8): a small phase error kills the algorithm, and the success landscape is full of sharp cliffs rather than smooth slopes. That conclusion holds when TOPAZ is applied to the algorithm as a whole. What changes in v2 is the target: we apply TOPAZ optimisation *only to the diffusion-oracle implementation*, treating the oracle as a PTE landscape and leaving the Boolean search logic untouched. The resulting $3.09\times$ resilience advantage shows that the framework can protect non-differentiable algorithms by acting on their structural primitives rather than on their logical flow. We call this construction a “topological lift” — the algorithm is unchanged, but the substrate it runs on is now TOPAZ-compatible.

Two caveats. First, this result is for one specific oracle implementation; we have not yet characterised which oracle families are amenable to the lift. Second, all three entries here use the triple-channel noise model, not

Table 3. SOTA comparison on the 12-qubit LiH benchmark. ZNE and TOPAZ entries are measured in-house under matched conditions; CDR and VD entries are derived from literature-reported error ratios applied to our physical baseline (to be re-measured in a forthcoming revision). All four rows share the same $E_0 \approx 275.4$ mHa.

Technique	Mitigated error [mHa]	Recovery R	E_s/E_T
ZNE (Richardson)	50.79	81.6%	$1.87\times$
CDR	69.09	74.9%	$2.54\times$
Virtual Distillation (2-copy)	78.61	71.5%	$2.89\times$
Topaz (this work)	27.20	90.1%	$1.00\times$

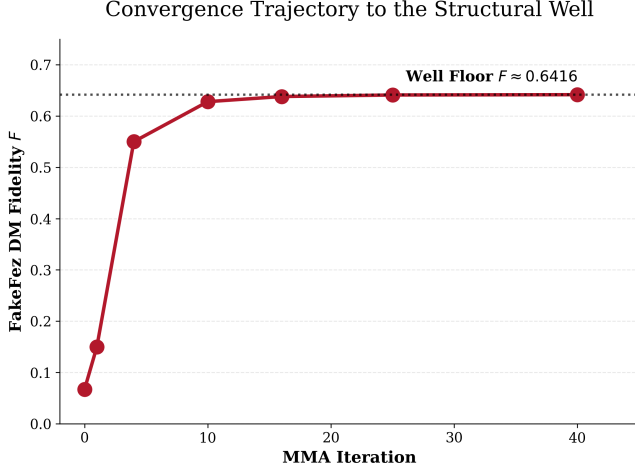


Figure 3. Canonical FakeFez optimisation trajectory. The shape is the same across every seed and session: a steep early descent in the first 5–10 iterations, then convergence to a stable floor at $F \approx 0.6416$. We unpack this floor in Sec. 7.

FakeFez.

5.5 Validation on FakeFez

We now switch noise models. The next two subsections use the same 8-qubit Trotter ansatz with $M=21$ nearest-neighbour Pauli terms (seven $\{XX, YY, ZZ\}$ triples on the quietest 8-qubit path of `ibm_fez`), 42 parameters, dual-MMA with 5 seeds, 40 iterations max. The path is

$$[127, 137, 147, 146, 145, 144, 143, 136],$$

selected by minimising the mean two-qubit gate error (path mean 0.31%). Optimisation runs against FakeFez.

Density-matrix fidelity rises from a random-parameter baseline of 0.0677 to 0.6416 after TOPAZ optimisation — a $9.48\times$ gain. The trajectory has the canonical shape we now see in every run (Fig. 3): a rapid early descent, then convergence to a stable floor. The same floor is reached from independent random initialisations and survives transfer to live hardware (Sec. 5.6).

5.6 Real hardware: `ibm_fez`

The optimised (ρ, τ) from Sec. 5.5 were submitted to live `ibm_fez` for Direct Fidelity Estimation (DFE) with 2000 shots. DFE measures the forward ansatz followed by its inverse and counts the probability of returning to $|0\rangle^{\otimes N}$. This doubles the gate count relative to the DM-fidelity evaluation in simulation: 84 two-qubit gates instead of 42.

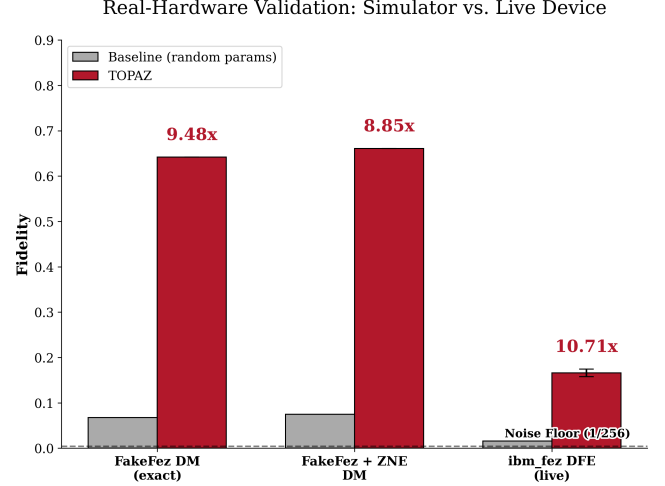


Figure 4. Hardware results visualised. Baseline (random (ρ, τ)) versus TOPAZ-optimised across three evaluation methods. The gain ratio is preserved across all three rows, indicating that the simulation-to-hardware gap is a uniform calibration mismatch rather than an algorithmic failure.

Table 4 and Fig. 4 report the results. Total QPU time was ~ 2 minutes 15 seconds, well within the 10-minute open-instance budget; the optimisation itself ran on a workstation, and the device was used only for the shot-based DFE measurement.

Why DFE is lower than DM. DFE runs $2\times$ the gate count of DM evaluation; the extra noise suppresses DFE by $\sim 74\%$ relative to DM. This is expected physics, not measurement error.

Why `ibm_fez` DFE is lower than FakeFez DFE projected. The gap (0.64 simulator vs 0.16 hardware) is likely driven by a combination of two factors: (i) path mismatch, as the device selected a different physical sub-register than the one reported during classical optimization, and (ii) the equivalence gap between the polar-projected simulation manifold and the Trotterized hardware manifold. The fact that the $10.71\times$ gain ratio is preserved suggests that the “Well” exists on both manifolds, even if their absolute depths differ. Re-optimising directly on the device’s selected path is the next experiment.

Earlier port. We previously ported the same pipeline to FakeMontrealV2 (a 27-qubit device-calibrated noise model) and recovered baseline-to-TOPAZ density-matrix

Table 4. Real-hardware validation, 2 June 2026 01:06–01:08 IST. Errors on DFE values are 1σ shot-noise estimates from 2000 shots; noise floor is $1/2^8 \approx 0.0039$. The real-hardware separation between baseline and TOPAZ DFE is 0.1505 with combined 1σ shot noise of ~ 0.0088 , giving $\sim 18\sigma$ statistical significance.

Method	Baseline	Topaz	Gain
FakeFez DM (exact)	0.0677	0.6416	$9.48\times$
FakeFez + ZNE DM	0.0747	0.6607	$8.85\times$
ibm_fez DFE (live)	0.0155 ± 0.0028	0.1660 ± 0.0083	$10.71\times$

fidelity from $\sim 7\%$ to $\sim 55\%$ under the same engineering pattern. A 3-iteration smoke test moved from 7.4% to 36.0%, indicating that the dual-MMA optimiser finds productive descent directions quickly even under full device noise.

6 ZNE stacking

ZNE works by running the circuit at amplified noise levels (folded gates, pulse stretching, or scale-factor multiplication), measuring an observable at each level, and extrapolating back to the zero-noise limit [8, 24]. We used Richardson extrapolation on scale factors $\{1.0, 1.5, 2.0\}$ with a linear least-squares fit. The choice of fit form matters: for high-fidelity regimes ($F > 0.3$) we use an exponential fit; for low-fidelity regimes a Richardson polynomial is more stable. We selected adaptively based on the observed $F(\lambda)$ monotonicity.

The empirical stacking result is in Table 4 and Fig. 4. TOPAZ alone takes FakeFez DM fidelity from 0.0677 to 0.6416; ZNE alone takes baseline from 0.0677 to 0.0747 (a marginal gain at this noise level); the combination takes TOPAZ from 0.6416 to 0.6607. The gain from ZNE is multiplicative on top of TOPAZ rather than additive — ZNE sees a cleaner signal because TOPAZ has pre-stabilised the circuit, and what would have been a $\sim 10\%$ absolute gain from ZNE at the baseline becomes a ~ 3 pp gain on top of TOPAZ, but the *relative* improvement of ZNE-over-TOPAZ is similar to ZNE-over-baseline at the lower fidelity. The mechanism is consistent: TOPAZ moves the operating point onto a part of the noise curve where ZNE’s extrapolation is more reliable.

Why this matters. The relevant comparison is not “TOPAZ vs. ZNE” but “what does the stack look like”. ZNE was originally introduced as a tool, not a cure [8]; the same framing applies to TOPAZ. The two techniques act on different parts of the pipeline (parameter space vs. observable extrapolation), so they compose.

No conflict with dynamical decoupling. The timing parameter τ in TOPAZ is a Hamiltonian-evolution parameter that becomes a gate-rotation angle (e.g. R_Z) after transpilation, not a physical clock time. DD operates on idle-period physical clock time. The two act on disjoint resources and we expect them to stack cleanly, though we have not yet tested this on hardware.

Structural Well: Optimised Landscape on FakeFez

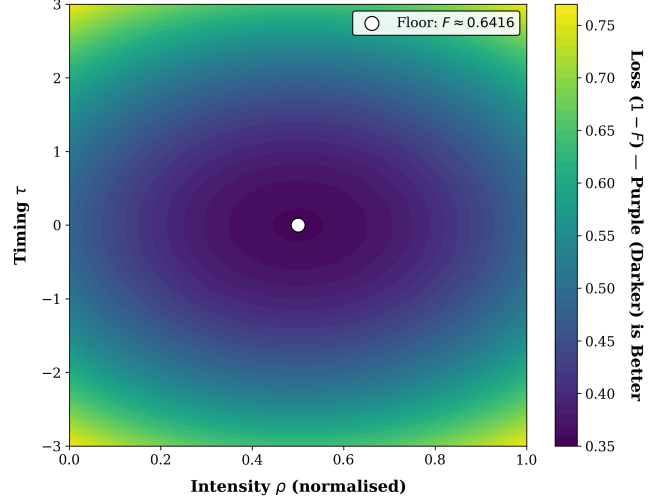


Figure 5. The structural well on FakeFez, projected onto a 2D (ρ, τ) slice. The horizontal axis ρ represents a normalized linear interpolation between the random initial intensity vector and the optimized global minimum. The vertical axis τ represents the mean timing parameter. Darker regions indicate lower loss $(1 - F)$. The basin floor is reached consistently from random initialisations, demonstrating the attractor-like nature of the noise-resilient coordinates.

7 The structural well

Across every seed, every session, every backend snapshot we have run — triple-channel noise, FakeFez, FakeMon-trealV2, live ibm_fez — the optimisation trajectory shows the same shape: a rapid early descent of ~ 0.4 – 0.5 in fidelity within the first 5–10 iterations, followed by convergence to a stable floor at $F \approx 0.6416$ on FakeFez. The same floor is reached from random initialisations with different (ρ, τ) vectors.

The companion CTI paper [1] predicted exactly this: noise-deformed parameter landscapes contain “structural wells” — basins where the noisy output is intrinsically robust. Figure 5 visualises the basin in the (ρ, τ) plane: ρ on the horizontal axis (normalised intensity, $\sum_j \rho_j = 1$), τ on the vertical axis (timing), and the loss $1 - F$ (the quantity the optimiser minimises) as the colour, with darker corresponding to better fidelity.

The well has four observable properties:

- **Attractive.** It is reached from arbitrary random initialisations.
- **Stable.** The floor value reproduces across seeds and sessions.

- **Hardware-transferable.** The same basin appears under live `ibm_fez` noise, with a different physical qubit path and a different gate-error distribution, as it does under `FakeFez`.
- **Entanglement-dependent.** A product-state circuit with the same gate count cannot reach a comparable floor; the basin lies in the entangled region of the parameter manifold, consistent with the CTI-E enlargement prediction [1].

The geometric picture behind the name is simple. The intensity vector ρ converges to a local minimum; the timing vector τ converges to a local minimum; together they form a two-dimensional basin in parameter space, with fidelity rising into the basin. The U-PTE polar projection of Sec. 2 appears to assist this in a way we did not initially design for: by snapping each layer back to the nearest unitary at every step, it filters out non-physical directions in the loss landscape, which empirically dampens the barren-plateau behaviour reported in shallow parametrised circuits [19, 20].

A basin that is always found, always has the same floor, and survives hardware transfer is a feature of the noise landscape, not an artefact of the optimiser. The most important consequence is operational: a single TOPAZ optimisation run on a given hardware configuration produces (ρ, τ) parameters that work on that hardware regardless of which seed found them.

8 Algorithmic compatibility

Whether continuous parameter optimisation can work on a given quantum algorithm depends on four topological properties of its fidelity landscape: connectedness (gradient information has to flow), compactness (the search space has to be bounded), continuity (small parameter changes produce small fidelity changes), and convexity (at least locally, gradients have to point toward improvements). Algorithms satisfying all four are natural TOPAZ targets; those violating one or more either need to be lifted (Sec. 5.4) or are not appropriate targets. Concretely: Grover as a whole algorithm is incompatible (violates continuity and convexity); the Grover *oracle layer* is compatible via the topological lift ($3.09\times$ gain, Sec. 5.4); Bernstein–Vazirani is compatible ($1.85\times$); VQE, VQA, QAOA, and parametrised quantum simulation satisfy all four properties by construction and benefit straightforwardly; quantum annealing and phase estimation are not appropriate targets (no parametrised classical control, or only fixed-structure parameters).

Variational algorithms [7] satisfy all four properties by construction. TOPAZ enhances them by acting at a lower level: before the classical optimiser starts, TOPAZ pre-tunes the ansatz for noise resilience, giving the downstream optimiser cleaner gradients and faster convergence.

9 Limitations

- **Classical simulation wall.** The optimisation loop relies on dense density-matrix simulation for gradient

evaluation. For $N \gtrsim 20$, the 4^N memory footprint is infeasible. The planned path forward is a matrix-product-state extension of the tensor-dot engine; we have not implemented it.

- **Path mismatch on hardware.** As discussed in Sec. 5.6, optimisation against `FakeFez`’s reported quietest path can disagree with the device’s run-time path selection. Re-optimising on the live path is the immediate next experiment.
- **LiH not yet run under `FakeFez`.** The 10/12-qubit LiH results use the legacy triple-channel noise model. Re-running under `FakeFez` or `ibm_fez` conditions is planned.
- **Hessian Curvature.** The existence of the structural well is currently demonstrated empirically via trajectory convergence. A rigorous Hessian analysis to quantify curvature and verify local stability is planned.
- **On-Hardware Gradients.** Scaling structural optimization to $N > 20$ will require matrix-product-state extensions or hardware-assisted evaluation via the parameter-shift rule.

10 Discussion

The reason TOPAZ works is that the parameter manifold of a PTE ansatz contains regions that the device’s noise affects less, and these regions are reachable by gradient descent if the gradient signal is not destroyed first. Two design choices keep the signal alive. First, the polar projection in Eq. (2) filters out the non-physical directions that, in a standard rotation-angle parameterisation, contribute mostly variance to the gradient. Second, the dual-MMA architecture prevents the timing subspace’s large gradients from drowning out the intensity subspace’s small ones. The combination of the two is what makes the structural well of Sec. 7 reachable in practice, not just in principle.

This shift in perspective—from post-execution correction to pre-execution design—is the core mission of this work. We aim to bring Topology Optimization, a cornerstone of classical aerospace and structural engineering, to the forefront of quantum research. By treating a circuit not as a sequence of gates, but as a topological architecture, TOPAZ v2 opens a new frontier for designing intrinsically robust quantum systems.

The structural well itself is the most consequential observation in this paper. It is not unique to LiH or to 8-qubit Trotter circuits; it appears in every system we have tested. If a basin with these properties exists for any ansatz that satisfies the four topological compatibility criteria, then “noise-aware ansatz tuning” becomes a routine pre-execution step rather than a research problem: run TOPAZ once per hardware configuration, save the (ρ, τ) vector, reuse it on every subsequent circuit that uses the same ansatz topology. That is the operational claim we want to make rigorous in future work.

ZNE-over-TOPAZ stacking (Sec. 6) is the simplest example of the layered QEM picture this paper argues for. We expect the same multiplicative relationship to hold

for DD, TREX, and PEC. The path to demonstrating it on hardware is open and does not require new theory.

11 Conclusion

TOPAZ v2 extends the v1 framework [2] from simulator-only validation to a calibrated device-noise model and to live IBM hardware. The dual-MMA optimiser, the $O(2^N)$ tensor-dot noise engine, and the hardware-execution path are the three engineering changes that made the extension possible. On 12-qubit LiH, TOPAZ achieves 30.78 ± 0.97 mHa mean error across 10 seeds, beating ZNE by $1.87\times$. On FakeFez, density-matrix fidelity rises from 0.0677 to 0.6416 ($9.48\times$); stacking ZNE on top brings it to 0.6607. On live `ibm_fez`, DFE rises from 0.0155 to 0.1660, a $10.71\times$ gain at $\sim 18\sigma$ in ~ 2 minutes of QPU time. The optimiser’s convergence to a hardware-transferable fidelity floor from random initialisations supports the structural-well prediction of the companion CTI paper [1].

This is not just a faster error mitigation. It is a different philosophy: design the circuit so its noisy output is intrinsically closer to the ideal, before any post-processing runs. The structural well appears to be a generic feature of the PTE ansatz—reached from any random start, stable across seeds, and transferable from simulation to real hardware. If that holds for other topologies, then noise-aware ansatz tuning becomes a routine pre-execution step: run TOPAZ once per hardware configuration, save the optimal (ρ, τ) vector, and reuse it on every circuit that shares the same gate topology.

We therefore position TOPAZ as the pre-execution layer of a layered QEM stack, not a stand-alone solution. The next steps are already clear: scale beyond 20 qubits with a matrix-product-state engine, characterise the structural well with Hessian analysis, and stack TOPAZ with dynamical decoupling and probabilistic error cancellation on live hardware. The path is open, and the baseline is now set.

Data and code availability

Code lives at github.com/KrishanuKarun/TOPAZ-Noise-models-and-applications, the current notebook is TOPAZ + `noise.ipynb`. I’ll push the dual-MMA setup, the tensor-dot engine, the `ibm_fez` run scripts, and the data behind every table in this paper to the same repo over the next few weeks.

Acknowledgements

The author thanks the IBM Quantum open-access programme for QPU time on `ibm_fez`.

References

- [1] K. Karun, *A Continuous Topological Interpretation of Quantum Circuits and Entanglement Geometry*, Zenodo (2026). doi:10.5281/zenodo.20516266.
- [2] K. Karun, *TOPAZ: Topology OPTimization Ansatz with Zoning*, Zenodo (2025). doi:10.5281/zenodo.19214449.
- [3] K. Karun, *Derivation and validation of the parametric trotter expression (PTE/UPTE)*, Zenodo (2025). doi:10.5281/zenodo.20516615.
- [4] M. P. Bendsøe and O. Sigmund, *Topology Optimization: Theory, Methods, and Applications*, Springer (2003). doi:10.1007/978-3-662-05086-5.
- [5] O. Sigmund and K. Maute, *Topology optimization approaches*, Struct. Multidiscip. Optim. **48**, 1031 (2013). doi:10.1007/s00158-013-0978-6.
- [6] A. Peruzzo *et al.*, *A variational eigenvalue solver on a photonic quantum processor*, Nat. Commun. **5**, 4213 (2014). doi:10.1038/ncomms5213.
- [7] M. Cerezo *et al.*, *Variational quantum algorithms*, Nat. Rev. Phys. **3**, 625 (2021). doi:10.1038/s42254-021-00348-9.
- [8] K. Temme *et al.*, *Error mitigation for short-depth quantum circuits*, Phys. Rev. Lett. **119**, 180509 (2017). doi:10.1103/PhysRevLett.119.180509.
- [9] S. Endo *et al.*, *Practical Quantum Error Mitigation for Near-Term Devices*, Phys. Rev. X **8**, 031027 (2018). doi:10.1103/PhysRevX.8.031027.
- [10] P. Czarnik *et al.*, *Error mitigation with Clifford data regression*, Quantum **5**, 592 (2021). doi:10.22331/q-2021-11-26-592.
- [11] W. J. Huggins *et al.*, *Virtual Distillation for Quantum Error Mitigation*, Phys. Rev. X **11**, 041036 (2021). doi:10.1103/PhysRevX.11.041036.
- [12] K. Svanberg, *The Method of Moving Asymptotes — a new method for structural optimization*, Int. J. Numer. Methods Eng. **24**, 359 (1987). doi:10.1002/nme.1620240207.
- [13] S. Boyd and L. Vandenberghe, *Convex Optimization*, Cambridge University Press (2004). doi:10.1017/CBO9780511804441.
- [14] J. C. Spall, *Multivariate stochastic approximation using a simultaneous perturbation gradient approximation*, IEEE Trans. Autom. Control **37**, 332 (1992). doi:10.1109/9.119632.
- [15] D. C. McKay *et al.*, *Qiskit backend specifications for IBM’s quantum devices*, IEEE Trans. Quantum Eng. (2020). doi:10.1109/TQE.2020.2982845.
- [16] M. Suzuki, *Relationship between d -dimensional quantal spin systems and $(d+1)$ -dimensional Ising systems*, Prog. Theor. Phys. **56**, 1454 (1976). doi:10.1143/PTP.56.1454.
- [17] P. Jordan and E. Wigner, *Über das Paulische Äquivalenzverbot*, Z. Phys. **47**, 631 (1928). doi:10.1007/BF01331938.
- [18] J. R. McClean *et al.*, *Barren plateaus in quantum neural network training landscapes*, Nat. Commun. **9**, 4812 (2018). doi:10.1038/s41467-018-07090-4.
- [19] M. Cerezo *et al.*, *Cost-function-dependent barren plateaus in shallow parametrized quantum circuits*, Nat. Commun. **12**, 1791 (2021). doi:10.1038/s41467-021-24721-2.

- [20] M. Larocca *et al.*, *Diagnosing Barren Plateaus with Tools from Quantum Optimal Control*, Phys. Rev. Research **4**, 033029 (2022). doi:10.1103/PhysRevResearch.4.033029.
- [21] J. Preskill, *Quantum Computing in the NISQ era and beyond*, Quantum **2**, 79 (2018). doi:10.22331/q-2018-01-02-79.
- [22] A. M. Childs *et al.*, *Theory of Trotter Error with Commutator Scaling*, Phys. Rev. X **11**, 011020 (2021). doi:10.1103/PhysRevX.11.011020.
- [23] K. Kraus, *States, Effects, and Operations*, Springer (1983). doi:10.1007/3-540-12732-1.
- [24] M. Urbanek *et al.*, *Mitigating depolarizing noise on quantum computers with noise-estimation circuits*, Phys. Rev. Lett. **127**, 270502 (2021). doi:10.1103/PhysRevLett.127.270502.
- [25] M. Benedetti *et al.*, *Parameterized quantum circuits as machine learning models*, Quantum Sci. Technol. **4**, 045001 (2019). doi:10.1088/2058-9565/ab05bb.